Time Perception from Interacting Oscillators

An Overnight Report

Exploratory session

2026-06-11

What this project is

The goal is a small, self-contained model of *subjective* time — the feeling of how much time has passed — built entirely out of **oscillators** and tested as a working program. No language model is involved.

An **oscillator** here is just something that cycles steadily, like the second hand of a clock going round and round. Two numbers describe it at any moment: its **frequency** (how fast it goes round — cycles per second) and its **phase** (where in the current cycle it is right now, which we can think of as an angle from 0° to 360°, or equivalently a fraction from 0 to 1). A collection of such oscillators is a "bank."

The central idea is that a bank of oscillators running at different speeds can act as a clock, and that the *feeling* of time can be read off that clock in different ways depending on what you ask. This report describes what was built on top of the Stage 1 engine, what the experiments showed (including two places where the first guess was wrong and had to be corrected), and where to go next.

1 The six main results, in one paragraph each

1. The "holiday paradox" works, and it is not fragile. The same system makes an idle stretch of time feel *long* while you are living through it, yet makes a busy stretch be *remembered* as having lasted longer. These two facts point in opposite directions, and both fall out of the model. Crucially, this is not a fluke of one lucky setting — it holds across a wide range of conditions (Section 3).

2. The scale-invariance of timing errors needs a specific kind of noise. Human time estimates get *proportionally* more variable for longer intervals (your error timing 10 seconds is about ten times bigger than your error timing 1 second). This appears only when the random fluctuation in the clock is *shared* across all the oscillators at once, not when each oscillator wobbles on its own. The kind of randomness matters, not just the amount (Section 4).

3. The textbook "psychophysics" of timing does not come from the oscillators themselves — it needs an extra step. Two classic laboratory findings about time perception turned out to require representing time on a *ratio* (logarithmic) scale, which the raw oscillator machinery does not do by itself. Oscillations are necessary but not sufficient (Section 4).

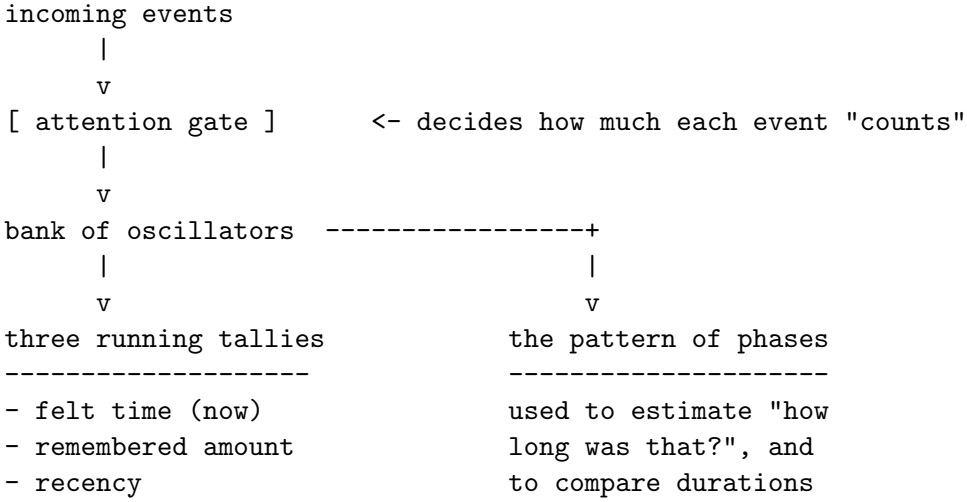
4. Making the oscillators influence each other helps only a little, and too much of it is catastrophic. When the oscillators pull on one another, a small amount improved one useful signal, but past a tipping point they all fell into lockstep and the clock stopped being able to tell different times apart (Section 5).

5. There are two ways to build a multi-speed clock, with opposite weaknesses. One is compact and exact but shatters under the slightest noise; the other is bulky and approximate but degrades gracefully. This is a genuine engineering trade-off between *capacity* and *robustness* (Section 6).

6. That trade-off can be fixed by being selective about which oscillators talk to each other. Letting each oscillator influence only its immediate neighbour (rather than all of them at once) keeps the compact clock from shattering, without causing the lockstep collapse from result 4. *Who* is coupled to *whom* matters more than how strongly (Section 7).

2 How the model is put together

There is one engine and several different "read-outs" — different ways of interrogating the same underlying state.



The pieces, in plain terms:

- **Event:** something happening — a stimulus arriving, a message, an action. In the model an event gives the oscillators a little nudge.
- **Attention gate:** a control that decides how strongly incoming events register. When a lot is going on, attention is absorbed by the events and the gate to the internal clock partly closes; when little is going on, the gate is open and the clock is watched closely. (This is the standard explanation for why a watched pot never boils — when you attend *to time itself*, it feels slow.)
- **Felt time ("now"):** a running tally of subjective time as it is being lived.
- **Remembered amount:** a running tally of how much happened, used when looking *back* on a stretch of time.

- **Recency:** a signal that fades as time passes since the last event, letting the system sense "how long since something happened."

3 The holiday paradox

The headline result. Take two stretches of clock time, each 60 seconds long. In one ("busy") many events happen; in the other ("idle") almost nothing happens. The model reports:

What we measure	Busy stretch	Idle stretch	Meaning
Felt time, as lived	∼6 seconds	∼43 seconds	the idle stretch **drags**
Remembered amount	high (∼4300)	low (∼340)	the busy stretch is recalled as **longer**

So the busy stretch *flies by* in the moment but is *remembered as long*; the idle stretch *drags* in the moment but *shrinks to almost nothing* in memory. This is the everyday "holiday paradox": an absorbing holiday races past while you are on it, yet feels long when you look back, because so much happened. The model reproduces both halves because the two read-outs are driven by opposite things — felt time runs fast when little is happening (attention is free to watch the clock), while the remembered amount simply counts up how much occurred. This is the reason the system is more than a stopwatch: a stopwatch would give the same answer both ways.

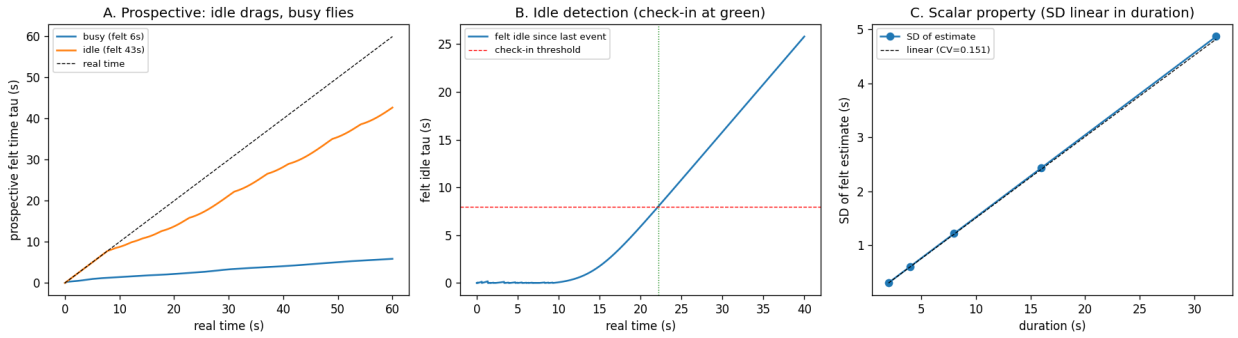


Figure 1: Left: felt (prospective) time vs. real time — the idle stretch tracks near real time (drags), the busy stretch lags far behind (flies). Middle: a "check-in" fires when *felt* idle time crosses a threshold. Right: the scale-invariance of error (see Section 4).

3.1 Is it just a lucky setting?

A fair worry: maybe the paradox only appears because of two convenient cases. To check, the event rate was varied smoothly from almost-nothing to very-busy.

- Felt time fell *smoothly and continuously* as things got busier (from ∼55 down to ∼3 seconds).
- Remembered amount rose *smoothly and continuously* over the same range.

The two curves cross and keep pointing in opposite directions across the whole range, and this stayed true for every setting of the gate's two tuning knobs that was tried. In other words, the effect is a broad, stable regime, not a coincidence that holds only at one exact balance point. (A "knife-edge" result is one that only works at a single precise tuning and vanishes if you nudge it; this is not one.)

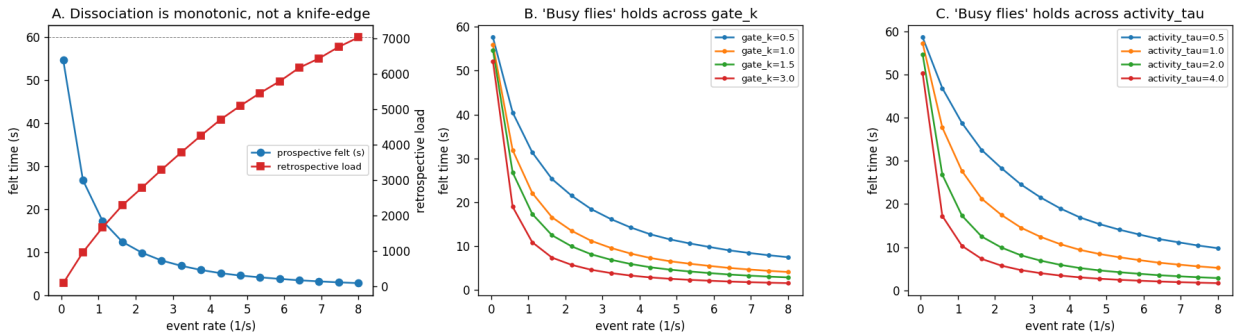


Figure 2: Left: as events get more frequent, felt time (blue) falls while remembered amount (red) rises — the dissociation holds across the whole range. Middle and right: "busy flies" holds for every setting of the two gate parameters; they only change the steepness.

4 Two classic timing laws — and where they actually come from

Psychologists have catalogued several robust facts about how people judge durations. Three were tested directly against the oscillator clock. One came out naturally (with a caveat); two did **not**, and understanding why was the most useful part of the work.

Two definitions used throughout:

- **Coefficient of variation (CV):** the spread of a set of measurements divided by their average. It is *relative* error. A CV of 0.1 means the typical error is 10% of the thing being measured, whatever its size.
- **Noise / jitter:** small random variation added to the model. Two kinds. *Shared* (common-mode) jitter means all oscillators speed up or slow down together on a given trial — like the whole clock running slightly fast that day. *Independent* jitter means each oscillator wobbles on its own, unrelated to the others.

4.1 Scale-invariance of error ("Weber's law")

The fact: a person's error in judging a duration grows in *proportion* to the duration, so the *relative* error (the CV) stays roughly constant across durations. This proportional-error property is called the **scalar property** or **Weber's law**.

- With **shared** jitter, the CV stayed flat at about 0.05 across durations from 2 to 32 seconds. That is exactly Weber's law, and it matches the textbook account in which the clock's *overall rate* varies a little from trial to trial.
- With **independent** jitter, the CV did *not* stay flat — it grew from 0.03 to 0.23 as durations got longer. Each oscillator drifting on its own scrambles the clock's pattern at long durations rather than simply rescaling it.

The lesson: Weber's law pins down the *kind* of randomness in the clock. It must be a shared, whole-clock fluctuation, not independent per-oscillator wobble.

4.2 Comparing two durations ("bisection")

The task: a person learns a "short" reference (say 4 seconds) and a "long" reference (16 seconds), then judges whether a new probe duration is closer to short or long. The interesting finding is *where the cross-over sits* — the probe judged "long" half the time. In people, it lands near the **geometric mean** of the two references.

The geometric mean of two numbers is the square root of their product: $\sqrt{4 \times 16} = 8$. It is the natural midpoint on a *ratio* scale (8 is the same factor ($\times 2$) above 4 as 16 is above 8), as opposed to the ordinary **arithmetic mean**, $(4 + 16)/2 = 10$, which is the midpoint on an additive scale.

The raw oscillator comparison **fails** at this task. With only two stored references, a probe sitting between them is too far from *both* to match either, and the decision collapses into noise (the cross-over landed at a meaningless $\sim$4.7 seconds). The reason is that the oscillator bank's ability to distinguish two times is set by a *fixed absolute resolution* (governed by its fastest oscillator) — it has no built-in sense of *ratios*, which is what the geometric mean is about.

The cross-over only lands correctly (at 8.0) once a read-out is added that represents time on a **logarithmic scale** — i.e. by the *logarithm* of the duration rather than the duration itself. On a log scale, equal *ratios* become equal *distances*, so the ordinary midpoint of the log values is the geometric mean of the durations. This is the extra step the oscillators do not provide on their own.

4.3 Estimates pulled toward the middle ("Vierordt's law")

The fact: when judging a range of durations, people overestimate the short ones and underestimate the long ones, as if every estimate is pulled toward the middle of the range. This central-pull is called **Vierordt's law**.

The standard explanation is **Bayesian**: a sensible observer combines a noisy measurement with prior expectations about what durations are likely, and that prior pulls estimates inward. Adding this Bayesian step on top of the log-scale read-out reproduces the law cleanly. Plotting estimated against true duration gives a line shallower than reality (a slope of 0.79 instead of 1.0; less than 1 means the estimates are compressed toward the centre), with the cross-over from over- to under-estimation sitting right at the middle of the range.

What these add up to. The oscillator bank supplies the multi-speed clock and, with shared jitter, the proportional error of Weber's law. But the *ratio-based* phenomena — geometric-mean comparison and central-pull estimation — live in a logarithmic read-out layered on top, not in the oscillators themselves.

5 Letting the oscillators influence each other

So far the oscillators run independently. The natural next question is what happens when they *interact*, each pulling on the others. The standard textbook model was used (**Kuramoto coupling**: every oscillator is nudged toward the average phase of the group, with a strength called K). Two new terms:

- **Coherence (order parameter):** a single number from 0 to 1 measuring how aligned the oscillators are. 0 means scattered all around the cycle; 1 means all bunched at the same phase, moving as one.

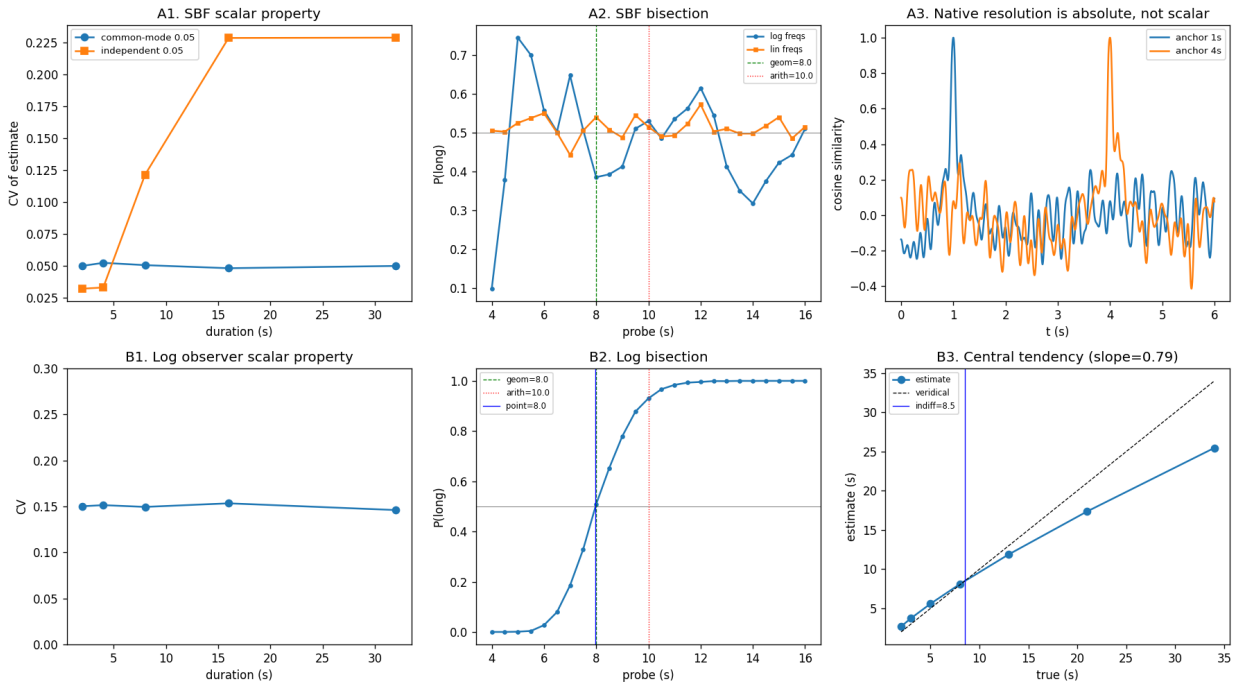


Figure 3: Top row, the raw oscillator clock: (A1) the CV is flat only with shared noise; (A2) it cannot do the comparison task; (A3) its resolution is a fixed absolute width, with no sense of ratio. Bottom row, the logarithmic read-out: (B1) flat CV, (B2) cross-over at the geometric mean, (B3) central-pull estimation.

- **Synchronization:** what happens at strong coupling — the oscillators stop running at their own speeds and fall into lockstep at a common speed.

Three things were measured as the coupling strength K increased:

- **The synchronization tipping point.** Up to about $K \approx 8$ the oscillators stayed mostly independent (low coherence); above it they snapped into lockstep (coherence jumped toward 1).
- **How long the clock stays useful (its "coding horizon").** After an event resets the oscillators to a common start, the pattern of phases is unique for a while, then eventually repeats — and once it repeats, the clock can no longer tell those two times apart. The time until that first repeat is the **coding horizon** (longer is better). Below the tipping point it was long (beyond the 30-second test window). At the tipping point it *collapsed* — from over 30 seconds to under half a second — because once everything is in lockstep at one speed, the pattern just repeats every cycle.
- **The recency signal.** Right after an event the oscillators are aligned (coherence near 1), and coherence then fades as they drift apart — that fading *is* a "time since last event" signal. Without coupling it faded very fast (coherence only 0.10 two seconds later). A little coupling made it fade more slowly. But strong coupling froze it near 1 forever, which is useless — a signal that never changes tells you nothing.

Verdict. Coupling earns its keep only in a narrow window of *weak* coupling (around $K \approx 6$–8), where it stretches the recency signal to last about three times longer while the clock is still intact. Push past the tipping point and the whole multi-speed clock collapses into a single useless rhythm.

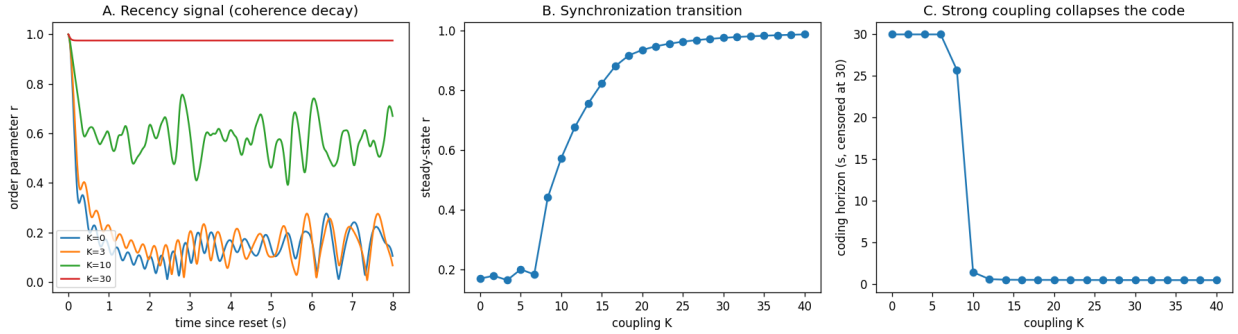


Figure 4: Left: the recency signal (coherence after a reset) for several coupling strengths — too little fades instantly, too much never fades. Middle: the synchronization tipping point near $K \approx 8$. Right: the coding horizon collapses at the same point.

6 Two ways to build a multi-speed clock

There are two quite different ways to get a clock that works across many timescales from a handful of oscillators, with opposite strengths and weaknesses.

The cascade (a stack of gears). Stack oscillators like the gears of a mechanical clock or the wheels of a car's **odometer**: each one runs exactly 10 times slower than the one below it. The fast wheel gives the fine detail; each slower wheel adds another digit. This is wonderfully compact: just 4 such wheels cover a 1000-second range *exactly*, the way 4 odometer wheels count to 9999. A linear number of oscillators buys an exponential range.

The incommensurate bank (many unrelated speeds). Alternatively, use a *bank* of oscillators whose frequencies are **incommensurate** — they share no common beat, e.g. $1.0, 1.7, 2.9$ cycles per second rather than the neat $1, 10, 100$ of the gears. No single one repeats in step with the others for a long time, so their combined pattern stays unique over a long span.

Both were tested against noise (random error added to each oscillator's phase):

- The **cascade** is *exact but brittle*. With no noise it is perfect. But the tiniest disturbance is catastrophic: like a slipped odometer wheel, a small error in a slow ("high-digit") oscillator throws the reading off by a huge amount. Just 1% phase noise already produced 1-second errors; 2% produced 10-second errors.
- The **incommensurate bank** is *approximate but graceful*. Over the same range and the same noise, its error stayed tiny throughout (a few hundredths of a second). The cost is that it needs more oscillators and covers a shorter unambiguous range.

So: **cascade = compact, exact, fragile; bank = bulky, approximate, sturdy.** This is a real capacity-versus-robustness trade-off, and it is the same lesson as the coupling result — the arrangement that packs in the most capability is also the most easily broken.

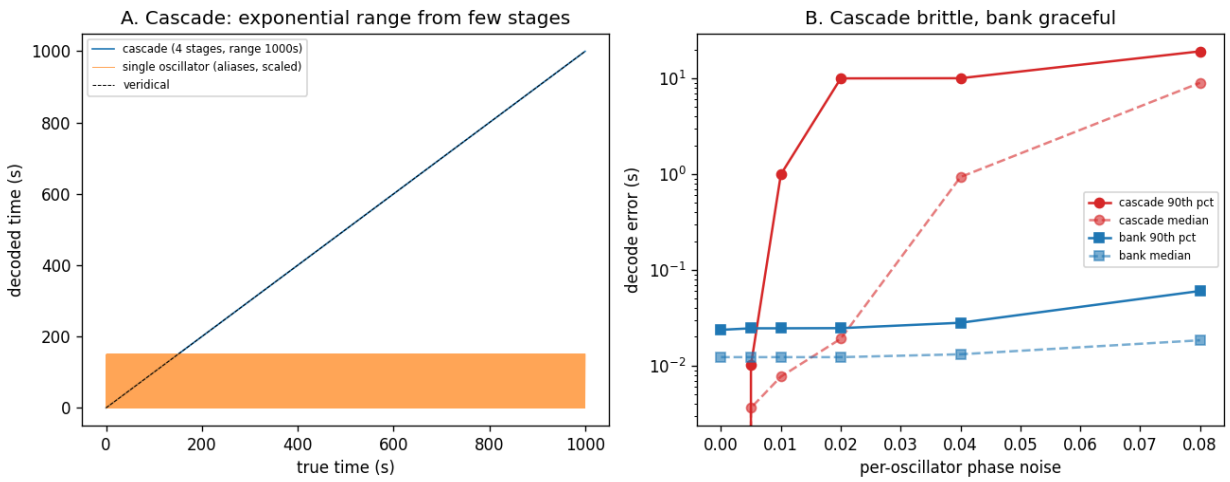


Figure 5: Left: the cascade decodes a long range exactly, where a single oscillator just repeats (aliases). Right: under increasing phase noise the cascade's error explodes (digit slips) while the incommensurate bank stays tiny.

7 Fixing the trade-off: be selective about coupling

Section 5 showed that coupling *all* the oscillators together destroys the clock. Section 6 showed the compact cascade is fragile. The resolution: couple the oscillators, but only **locally** — let each gear lock onto just its immediate faster neighbour, keeping the exact 10:1 ratio between them, rather than everyone pulling on everyone.

The fast oscillator is the most reliable (it completes many cycles, so small drifts average out), and each slower one can be gently kept in step with it, correcting drift before it causes an odometer-style digit slip.

Testing with a 3% speed drift added to each stage over a 60-second run:

- **No coupling:** typical error 7.25 seconds (the digit-slips above).
- **Local neighbour-coupling:** typical error 0.08 seconds — about a 90-fold improvement.
- **Even very strong local coupling:** still 0.08 seconds — it does *not* collapse the clock, unlike the all-to-all coupling of Section 5.
- **Tolerance to drift:** with no coupling, error grew to ∼18 seconds at 12% drift; with local coupling it stayed under a third of a second throughout.

This is the satisfying synthesis: the brittle-but-compact cascade becomes *robust* when stabilised by *local* coupling. The thing that matters is not how strong the coupling is but *who is coupled to whom* — neighbour-to-neighbour coupling preserves the ladder of speeds, whereas all-to-all coupling erases it. (One honest blemish: occasional ∼4-second glitches remain at the exact moments a digit rolls over.)

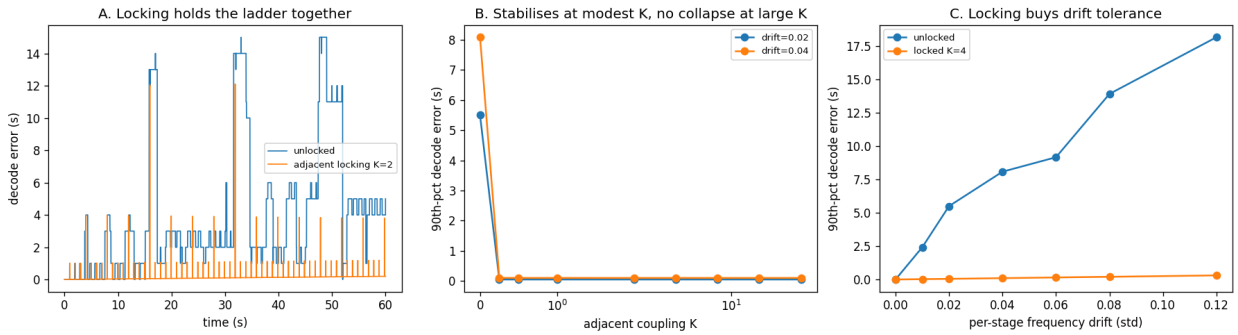


Figure 6: Left: decode error over time — unlocked drifts and jumps, local locking stays flat. Middle: error drops sharply at modest coupling and, unlike global coupling, does not collapse even when strong. Right: local locking tolerates far more drift.

8 The self-monitoring loop

A small but pleasing extra. The system raises a "check-in" flag after enough *felt* idle time. Feeding each check-in back in as if it were an event closes the loop and produces a self-sustaining rhythm — in the language of dynamical systems, a **limit cycle** (a stable repeating pattern the system settles into on its own):

- Left alone with nothing happening, the system checks in at very regular intervals (every 9.8 seconds, extremely steady).
- The interval (9.8 s) is a bit longer than the raw threshold (8 s), because each check-in briefly busies the system and partly closes the attention gate, so felt time accrues slowly until things settle — the period is an *emergent* compromise, not simply the threshold.
- Background activity *suppresses* the rhythm smoothly: the busier things are, the less often the system checks in (down to never, once enough is going on).

This is a concrete, dynamic version of the "nested feedback loops" idea: a rhythm the system generates itself and that its context modulates.

9 Honest limitations

- **Weber's law is partly built in, not discovered.** The proportional error comes from a shared, whole-clock fluctuation put in by hand. It is the standard and correct modelling choice, but an assumption, not something the dynamics produced on their own.
- **The ratio-based laws are not oscillatory.** Geometric-mean comparison and central-pull estimation come from the logarithmic Bayesian read-out, not from the oscillators. The pure "everything is oscillations" ambition hits a genuine wall here.
- **The "remembered amount" is a crude stand-in** (a sum of how much each event disturbed the clock). It gets the *direction* of the holiday paradox right but is not calibrated against real data.
- **The arousal/attention effect is a knob, not an explanation.** Turning it shifts estimates the right way, but that only confirms the dial is wired up sensibly.

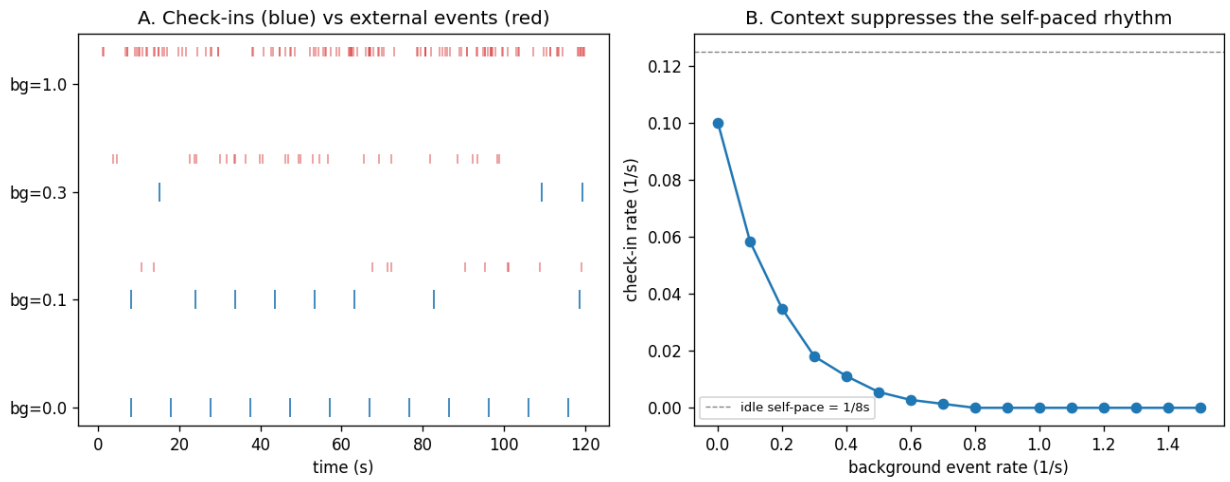


Figure 7: Left: check-ins (blue) against external events (red) at several background rates — regular when idle, suppressed when busy. Right: the self-check rate falls smoothly as the background gets busier.

- **The coupling experiments use a moderate spread of speeds** so the tipping point is reachable in simulation. The qualitative result does not depend on that choice, but the specific numbers do.

10 Where to go next (in priority order)

1. **A fully self-correcting clock.** Combine the local-coupling stabiliser with a *noisy fastest oscillator* (right now the fast one is assumed perfect) and add a little redundancy to remove the digit-rollover glitches.
2. **One unified pipeline.** Wire the oscillator clock, the logarithmic read-out, and the Bayesian step into a single object, so one model produces all the timing behaviours together.
3. **Couple the self-check rhythm to the "remembered amount,"** so that a *full* stretch of time changes the rhythm, not just an empty one.
4. **Ground "how much an event counts" in real surprise** (how unexpected an input is) if and when this is reconnected to a language model.

11 One decision for you

The two clock designs are genuinely different bets:

- The **cascade** (stacked gears) is compact, digital, and closest to the original "nested loops at different scales" image — and Section 7 shows it can be made robust.
- The **incommensurate bank** (many unrelated speeds) is sturdier out of the box but bulkier and shorter-range.

Which one to lean into shapes the next stage. The mild preference here is the cascade, now that local coupling has made it robust — it is the more elegant and more faithful to the starting idea

— but it is your call.